

Audit Catalog

This document catalogs all G-code audit checks the engine performs, plans to perform, or is considering. It serves as both a planning tool and a status tracker.

Status Legend

- `[x]` — **Implemented** — validator exists, tested against at least one real G-code file
- `[~]` — **Partial** — basic detection works but has known gaps or simplifying assumptions
- `[ ]` — **Planned** — not yet implemented, on the roadmap
- `[?]` — **Under consideration** — value or definition not yet settled

Severity Tiers

- Tier 1 — Critical Safety** — risk of machine damage, tool breakage, or injury. Must be flagged loudly.
- Tier 2 — Program Correctness** — won't crash the machine but will produce wrong parts, fail mid-job, or rely on machine state assumptions.
- Tier 3 — Efficiency / Best Practice** — won't break anything, but identifies suboptimal or sloppy programs.
- Tier 4 — Structural / Meta** — about the program as a document rather than its machining behavior.

Tier 1 — Critical Safety

Status	Audit	Source File
<code>[x]</code>	Cut before safe Z established	<code>gcode_audit_v3_050226.py</code>
<code>[]</code>	Rapid into material (G0 with Z below clearance)	—
<code>[x]</code>	Cutting motion with spindle off	<code>gcode_audit_v3_050226.py</code>
<code>[]</code>	Plunge feedrate exceeds safe ratio	—
<code>[]</code>	Tool change without retract	—
<code>[]</code>	Arc with invalid geometry (I/J/R inconsistency)	—
<code>[]</code>	Coordinate values outside machine envelope	—

Cut before safe Z established `[x]`

First $Z \leq 0$ motion happens before the program has ever moved Z above the work surface. Indicates the program may plunge into material or fixturing on startup. *Implemented in* `validate_startup_sequence`.

Rapid into material `[ ]`

A G0 motion that ends with Z below the clearance plane, or that moves XY while Z is below clearance. G0 is full machine speed — hitting material at rapid breaks tools and can crash the spindle. **#1 thing experienced machinists worry about.** Now possible because v3 tracks motion mode.

Cutting motion with spindle off `[x]`

G1/G2/G3 issued when spindle state is M5 or never started. Tool drags through material without rotating — instant tool breakage. *Implemented in* `validate_spindle_and_feed`.

Plunge feedrate exceeds safe ratio `[ ]`

A Z-only downward feed move (plunge) at the same feedrate as XY cutting. Plunge rates should typically be 30-50% of XY feed because end mills cut poorly straight down. Catches programs where the operator forgot to set a separate plunge feed.

Tool change without retract `[ ]`

M6 (tool change) issued while Z is below the safe plane. Most machines retract automatically but not all do — and assuming is how spindles get destroyed.

Arc with invalid geometry `[ ]`

G2/G3 where the radius from start to center doesn't match the radius from end to center (within tolerance). Indicates a corrupted I/J/R value. Many controllers will throw an error; some will execute a weird path. Worth catching before the machine sees it. Requires arc-resolution helper.

Coordinate values outside machine envelope `[ ]`

X/Y/Z values that exceed configurable machine travel limits. Requires a machine-config file as input. Catches misplaced decimal points

(`x1500` instead of `x150.0`) and wrong-units bugs (program written in inches but machine set to mm).

Tier 2 — Program Correctness

Status	Audit	Source File
[]	Missing modal declarations at startup	—
[]	Unit mismatch between header comment and G-code	—
[x]	Feedrate never set before first cutting move	<code>gcode_audit_v3_050226.py</code>
[]	Spindle speed never set	—
[x]	XY motion before safe Z	<code>gcode_audit_v3_050226.py</code>
[~]	Depth anomaly between operations	<code>gcode_audit_v3_050226.py</code>
[]	Incremental mode without absolute reset	—
[]	Work coordinate system never selected	—
[?]	Inconsistent decimal precision	—

Missing modal declarations at startup []

Program begins cutting without explicitly setting G90/G91 (distance), G20/G21 (units), or G17/G18/G19 (plane). Machine uses whatever mode was left over from the last program — recipe for surprise.

Unit mismatch between header comment and G-code []

Header comment says `{Units = mm}` but program uses G20 (inch), or vice versa. Catches post-processor configuration errors.

Feedrate never set before first cutting move [x]

G1/G2/G3 issued with no prior F-word. Behavior is controller-dependent; some refuse, some use a default that may be unsafe. Implemented in `validate_spindle_and_feed`.

Spindle speed never set []

M3/M4 issued without ever setting an S value. Same controller-dependent risk as missing feed.

XY motion before safe Z [x]

Implemented in `validate_startup_sequence`.

Depth anomaly between operations [~]

Operations expected to be at consistent depth show one outlier. *Partially implemented as `profile_operation_depths` (clusters by depth) — does not yet flag outliers as issues, only reports clusters. Needs a “depth outlier” rule on top of the profiler.*

Incremental mode without absolute reset []

Program enters G91 (incremental) and never returns to G90 before a section that looks like it expects absolute coordinates. Subtle and easy to miss in review. **Blocked on G91 coordinate interpretation (deferred from v3).**

Work coordinate system never selected []

No G54-G59 ever issued. Machine uses whatever WCS was last active. Programs should be explicit.

Inconsistent decimal precision [?]

Some lines use 3 decimal places, others 1, with no obvious pattern. Often indicates manually edited code or post-processor weirdness. Low-severity flag, mostly informational. *Under consideration — may be too noisy.*

Tier 3 — Efficiency and Best Practices

Status	Audit	Source File
[]	Redundant rapid moves	—
[]	Repeated tool path	—
[]	Non-monotonic depth progression	—
[]	Excessively long single moves	—
[]	Missing program end (M30/M2)	—

[]	missing program end (M30/M2)	—
[]	Coolant on without spindle	—
[]	Spindle still running at program end	—

Redundant rapid moves []

Consecutive G0 moves to similar coordinates, or rapids that overshoot and return. Often indicates post-processor inefficiency.

Repeated tool path []

Same XY path traversed twice with no Z change. Could be intentional (finishing pass) or a CAM bug.

Non-monotonic depth progression []

Operation goes deep, then shallow, then deep again. Sometimes intentional (rest machining) but often a CAM bug, especially when variation is small (e.g. -3.0 then -2.95 then -3.0).

Excessively long single moves []

A single G1 or G0 longer than a configurable threshold (e.g. 2× machine envelope diagonal). Almost always a typo.

Missing program end []

No M30 (or M2) at end of file. Most controllers handle it, but it’s poor practice.

Coolant on without spindle []

M7/M8 issued while spindle is off. Usually harmless but indicates programming sloppiness.

Spindle still running at program end []

File ends without M5. Some controllers handle it via M30, some don’t.

Tier 4 — Structural / Meta

Status	Audit	Source File
[]	Malformed tokens	—
[]	Comment density	—
[]	File length anomalies	—
[]	Block-skip characters present	—
[]	Line numbers (N words) inconsistent	—

Malformed tokens []

Tokens the parser couldn’t interpret. Currently the parser silently skips these — should be surfaced as informational issues. *Already flagged as deferred in v3 changelog.*

Comment density []

Programs with no comments at all in long sections. Informational; doesn’t affect execution.

File length anomalies []

Very short files (< 10 lines of code) or extremely long ones (above a configurable threshold) flagged for attention.

Block-skip characters present []

Lines starting with / are conditionally skipped depending on a control panel switch. Worth flagging because their behavior depends on machine state outside the file.

Line numbers (N words) inconsistent []

N-words present but non-sequential, or missing on some lines and present on others. Cosmetic but indicates manual editing.

Cross-Cutting Concerns

These aren’t audits per se, but design issues that affect multiple audits.

Machine configuration []

Many planned audits need parameters: safe Z height, machine envelope, max plunge ratio, default feed limits, etc. These should live in a per-machine config file, not be hardcoded. **Recommended scaffold task before implementing more than 2-3 additional audits.**

Severity overrides [?]

What’s CRITICAL on a hobby router might be WARNING on a production machine with crash protection. Worth designing the issue-output schema to allow severity overrides per audit per machine config. *Not urgent, but worth keeping in mind as audits multiply.*

Shared “machine context” object [?]

As validators multiply, a shared context object readable by all validators (rather than each one reading parser_output independently) may be cleaner. Defer until we have 5+ validators and an actual pain point to solve.

Notes for Domain Review

The audits above reflect general G-code knowledge and common-sense machining principles. **An experienced CNC operator would likely add audits not on this list, and disagree with some severity rankings.** Worth getting that perspective once we have a few more audits implemented and something concrete to react to. Particularly worth reviewing:

- Whether Tier 1 items match the operator’s actual “things that scare me” list
- Whether any common failure modes are missing entirely
- Whether the Tier 2/3 split matches operator priorities

Audit Completion Summary

- **Tier 1 (Critical Safety):** 2 of 7 implemented
- **Tier 2 (Correctness):** 2.5 of 9 implemented (one partial)
- **Tier 3 (Best Practice):** 0 of 7 implemented
- **Tier 4 (Structural):** 0 of 5 implemented
- **Overall:** 4.5 of 28 implemented

Last updated: 2026-05-02 (alongside v3 release)